

JANUS

A Personal Operating System of Isolated Worlds

Capability-based computing, persistent activities,
and explicit hardware ownership

Danyal A. Samak

dabsamak@tuta.com

www.cryogenix.org

ARCHITECTURE WHITEPAPER
VERSION 0.1 · DISCUSSION DRAFT
20 SEPTEMBER 2026

**A computer organised around its owner's activities,
not around ambient application authority.**

JANUS names the complete proposed operating system. LLMs are development tools, not required runtime components.

Status: architectural proposal. No implementation, benchmark result, certification, or whole-system verification is claimed. Kernel substrate, implementation language, and release licensing remain open.

Artwork note: the original JANUS logo was unavailable for this draft. This cover uses a typeset title, not a replacement logo.

Executive summary

JANUS proposes an experimental operating system for a personally owned computer. Its primary abstraction is the **world**: a bounded execution environment with explicitly granted authority, accounted resources, persistent state, and a declared relationship to physical devices. A world may be a small native protection domain, a lightweight hardware-virtualised environment, or an entire compatibility operating system. These forms share a management model without pretending to have identical isolation, cost, or checkpoint semantics.

The proposed system combines a capability-oriented supervisor with separate storage, display, input, networking, and device services. Applications receive access to selected objects and services rather than inheriting the owner's general authority. A persistent activity graph records what the owner was doing; a separate authority record determines what resumed software may do now. Device and foreground leases make “this computer is now this world” an explicit operation, while preserving a trusted route back to the owner's controls.

The central hypothesis is that this combination can deliver a more coherent personal computer: recoverable activities, understandable grants, predictable foreground responsiveness, and compatibility without making a legacy operating system the native architectural contract. This is a research hypothesis to test, not an established performance or security result.

LLM-assisted engineering is proposed for implementation, porting, documentation, and test development. Generated code receives no special trust. Independent tests, constrained build environments, human review, and narrowly scoped formal reasoning remain the acceptance mechanisms.

Reading map

Topic	Sections
Purpose, precedent, and system structure	1–3
Execution, authority, and persistent state	4–7
Hardware, presentation, networking, and compatibility	8–10
Software construction and assurance	11–13
Implementation, evaluation, and decisions	14–16
Illustrative contracts and terminology	Appendices A–B

Interpretation. “Proposed” identifies this paper's design choices. “Required” describes an intended JANUS invariant, not an implemented feature. Numbered references support external facts and architectural precedents; they do not certify JANUS.

1. Design intent and scope

1.1 A personal system, not an absence of protection

JANUS begins with one primary human owner and many mutually untrusted programs. The important boundary is therefore not chiefly between human login accounts. It is between activities, software suppliers, documents, peripherals, and services that should not automatically trust one another.

Single-owner does not mean that every component is privileged, that the machine remains unlocked, or that visitors and remote attackers disappear. JANUS still requires authentication, a lock state, encrypted storage, recovery credentials, and an explicit way to delegate limited access. Optional guest sessions can be worlds rather than additional users in a universal account hierarchy.

The owner is entitled to inspect, repair, replace, and develop the system. That authority should be exercised through a trusted control path, not inherited by every application the owner starts. A debugging grant may intentionally expose a chosen world's memory; it must not silently expose every other world.

1.2 The architectural departure

The proposal does not depend on declaring Unix obsolete or treating process isolation as inherently inadequate. It changes the native contract. A program is instantiated with a bounded set of capabilities and resources; a document is an object accessed through a grant; an activity is a persistent relationship among data, tools, and execution environments.

Paths, terminals, byte streams, and conventional files remain useful interfaces. JANUS need not prohibit them. It declines to make a machine-wide pathname hierarchy, an ambient owner identity, or a POSIX personality prerequisites for all native software. Compatibility worlds may retain these conventions internally.

1.3 Goals and non-goals

The first goals are understandable authority, recoverable work, an immediate foreground experience, inspectable system composition, and the ability to run existing operating systems. Responsiveness includes scheduling, storage contention, audio, input, and power management, rather than merely removing window decorations.

The initial scope is a locally usable experimental desktop on a documented hardware configuration. Universal laptop support, transparent distributed computing, arbitrary live migration, complete historical-machine fidelity, and a formally verified consumer software stack are not initial deliverables. Cloud accounts, an AI assistant, and an LLM inference service are not required to boot, recover, install local software, or use native applications.

“Post-Unix” describes the proposed application and authority model, not a claim that its individual mechanisms are unprecedented. JANUS should earn its identity through a working composition and measured behaviour, not through replacing familiar terminology alone.

2. Foundations and proposed contribution

JANUS combines established systems ideas rather than presenting virtualisation, capabilities, or persistence as new inventions. seL4 supplies a concrete precedent for a capability-based microkernel that can also serve as a hypervisor. Its verification claims apply under documented assumptions and to particular configurations, not automatically to every system assembled above it. [1, 2]

Genode provides a precedent for organising components, delegated resources, and application-specific trusted computing bases. Sculpt demonstrates a personal environment combining that architecture with sandboxed drivers and virtual machines. [3, 4] Qubes demonstrates desktop compartmentalisation using virtual machines and controlled interactions between compartments. [5]

Persistent capability systems also predate JANUS. CapROS, continuing the EROS lineage, explicitly combines capabilities with orthogonal persistence. Nemesis investigated time-sensitive applications and accounting across CPU, memory, network, and storage resources. [6, 7]

Precedent	Lesson for JANUS	Deliberate distinction
seL4	Small capability-oriented protection substrate	Kernel proofs do not establish the whole desktop's correctness.
Genode / Sculpt	Composable services and delegated resources	JANUS proposes a particular activity, persistence, and device-lease model.
Qubes	Compatibility environments as compartments	Native JANUS worlds need not contain a conventional guest OS.
EROS / CapROS	Persistence as a system concern	JANUS distinguishes durable data, runtime state, and current authority.
Nemesis	Resource accounting and responsiveness	Reservations must include service work performed for clients.

The proposed contribution is the integration of five contracts: a common world lifecycle; explicit, inspectable authority; a versioned activity/object graph; recoverable foreground and device ownership; and an engineering process that accepts machine-generated code only through evidence.

The strongest claim worth testing is not “a VM is a better process”. It is that applications and entire operating systems can participate in one owner-facing model while the system preserves the differences that matter underneath.

A JANUS implementation built on an existing capability framework would still constitute a distinct operating environment if it implements these contracts. A new kernel is justified only where a demonstrable requirement cannot be met by an existing substrate. Reuse is not an architectural failure, and a renamed virtual-machine launcher is not, by itself, the completed design.

The reference architecture below therefore leaves the native substrate open. The Linux-hosted prototype and any later microkernel implementation must make different trust claims explicitly, even where their user interfaces look identical.

3. System model and component boundaries

3.1 The world and the activity

A **world** is an independently managed execution and authority domain. Its durable identity is distinct from its current process identifier, memory image, or virtual CPU allocation. It has a manifest, a resource account, a set of granted endpoints, a lifecycle state, and an optional persistence contract.

An **activity** is an owner-facing grouping of worlds and objects: writing a document, developing JANUS, making music, or operating a historical workstation. An activity is not automatically a security domain. Related tools may remain isolated and receive different grants.

3.2 Reference decomposition

Layer	Proposed responsibilities
Personal environment	Activity shell; object picker; trusted grant/recovery interface; graphical and textual inspection
Application worlds	Native components; isolated library-OS environments; compatibility operating systems; emulators
System services	Object storage; policy and authority records; display/input/audio; networking; device brokerage; lifecycle management
Machine supervisor	Protection domains; capability enforcement; memory mappings; scheduling mechanisms; IPC; interrupts; virtual CPU and DMA isolation support
Platform	CPU; MMU; optional virtualisation extensions; IOMMU; firmware; physical peripherals

This is a responsibility map, not a requirement that every message cross every layer. A granted endpoint may connect two worlds directly. Large data transfers may use explicitly shared buffers while control messages carry bounds and ownership information.

The supervisor should be policy-minimal, not described as literally policy-free. Its object model, revocation rules, scheduling mechanisms, and treatment of faults necessarily encode choices. Filesystem formats, network protocols, desktop composition, and device-specific driver logic belong outside it.

3.3 The control plane must not become hidden root

Lifecycle control, grants, recovery, and key management are powerful functions. Merely moving them into services does not remove their authority. The design should separate a frequently used activity shell from the smaller trusted grant interface, update authority, and recovery mechanism. Routine navigation must not require unrestricted access to every document or secret.

Each release should publish a dependency and authority graph. For any important claim—such as document confidentiality—the graph identifies every component that can read the plaintext, alter its execution, or approve additional access. Genode’s application-specific TCB model provides a useful precedent for this analysis. [3]

At bootstrap, a narrowly defined initial authority allocates resources and starts the minimum trusted services. It then delegates or seals broad capabilities where the substrate allows. This reduces continuously exercised authority; it does not prove that no powerful component exists.

4. Execution, scheduling, and resource economics

4.1 One lifecycle, several mechanisms

Native worlds use ordinary hardware memory protection and capability-mediated communication. Lightweight virtualised worlds add a virtual CPU environment where that isolation or software packaging is valuable. Compatibility worlds include an existing operating system. Emulator worlds run software for another machine architecture inside a separately confined domain.

A world is therefore not synonymous with a VM. Hardware virtualisation is an available mechanism, not a mandatory tax on every parser or graphical widget. Native and VM-backed worlds should share lifecycle concepts while reporting their real execution class and security boundary.

QEMU distinguishes hardware accelerators from its Tiny Code Generator, which emulates CPUs. KVM exposes separate operations for creating VMs and virtual CPUs. These mechanisms demonstrate possible backends; neither is the proposed JANUS application interface. [8, 9]

4.2 Resource accounts

Each world receives limits or reservations for resident memory, CPU time, persistent storage, IPC queues, handles, and device work. Virtual CPUs are scheduling entities, not a promise of dedicated physical cores. An activity can have a parent budget divided among its worlds, but a child must not mint additional resources by creating descendants.

Shared services must charge work to the requester. A storage server or compositor that performs unlimited work for a nominally restricted client would defeat the accounting model. CPU budget donation, bounded queues, cancellation, and priority propagation should be evaluated as mechanisms, with explicit treatment of deadlock and priority inversion.

Foreground responsiveness should be protected by admission control and a reserved recovery budget. Background indexing, builds, backups, and VM activity may consume spare capacity but must not starve input, audio, the grant interface, or recovery. A missed reservation must be visible; hard real-time performance is not assumed on arbitrary consumer hardware.

4.3 Costs that must be measured

Virtualisation and service decomposition introduce costs through scheduling, address translation, copying, wakeups, device emulation, and inter-domain communication. Shared memory can reduce copying while expanding the trust relationship between participants. “Zero-copy” is not a security property.

The prototype should compare native protection domains, microVM-style worlds, and full compatibility guests under the same workloads. Measurements must include total service and supervisor memory, idle power, worst observed stalls, and tail latency—not only application throughput.

Memory overcommit should be conservative in the first reference system. World suspension is useful only when the declared persistence contract permits it; it must not be used as an invisible substitute for adequate memory or predictable admission decisions. A host-wide emergency must terminate or suspend low-priority work before destroying the owner’s recovery path.

5. Owner authority and capabilities

5.1 Authority follows a grant, not a name

A native application starts with no general authority over the owner's data, network, or peripherals. It receives specific capabilities: for example, a read handle to one image, a transaction endpoint for a selected collection, and a display surface. An object identifier or content hash is not a capability and must not be sufficient to obtain access.

Capabilities are unforgeable references enforced by the supervisor or an explicitly trusted service. Their rights should be attenuable: read rather than write, one object rather than a collection, one session rather than indefinite access. Their transfers occur through controlled IPC, not by copying arbitrary strings that happen to resemble handles.

A program that needs a photograph invokes an object picker through a supplied endpoint. The trusted picker returns the owner-approved capability. The program does not gain the picker's authority to enumerate unrelated collections.

5.2 Delegation and revocation

A grant record identifies its issuer, recipient, target, rights, delegation policy, and validity conditions. Revocable service access can use an indirection or epoch checked by the authority service. Memory and device grants also require the underlying mappings and operations to be withdrawn; removing a name from a directory is insufficient.

Revocation has a defined completion boundary. After it completes, new protected operations must fail. Work already committed may remain committed, and data already copied by a recipient cannot be made unknown again. Shared-memory revocation must address mappings, in-flight use, and relevant CPU/DMA translation state. The implementation must document which guarantees it actually provides.

5.3 The owner's trusted path

The activity shell may request a grant but must not impersonate the owner's approval. A small trusted interface identifies the requesting world, the exact authority change, and whether the grant survives suspension, restart, or an update. Dangerous combinations—such as private documents plus unrestricted network access—need a meaningful explanation rather than a stream of generic prompts.

The owner should be able to inspect and withdraw grants through a graphical interface or a capability-aware textual console. That console is not an ambient superuser shell; administrative actions still require explicit control capabilities.

A lock operation withdraws ordinary foreground input and access to protected key services according to policy. A recovery operation may deliberately supersede normal controls, but must require the configured owner authentication. The architecture does not equate one owner with one permanently trusted application session.

The invariant is bounded authority, not the metaphysical absence of privileged software. Components that authenticate the owner, distribute grants, or release keys remain in the relevant trusted computing base and require proportionately stronger review.

6. Persistent objects and the personal data graph

6.1 Separate identity, content, and access

JANUS proposes a transactional object service beneath owner-visible collections and folders. An object has a stable identity, a declared type and schema version, metadata, immutable content revisions, and references to related objects. A separate authority structure determines who may access those revisions.

Content addressing can identify immutable chunks and permit reuse of unchanged data. It does not, by itself, implement transactions, confidentiality, authorisation, or a coherent mutable identity. A document's stable identity may point to successive revision hashes; knowledge of either identifier must still be insufficient to read its content.

A transaction validates its expected base revision, writes new content, and atomically publishes a new root or revision record. Concurrent edits should produce an explicit conflict or an application-defined merge, never a silent overwrite presented as success. Multi-object atomic updates require a specified transaction boundary.

6.2 Files remain an interoperability format

Native software can use typed object interfaces or bounded byte-stream views. Imported directories, exported files, and guest filesystems are adapters rather than the sole authority mechanism. Native object features must not trap the owner's data: documented export formats and a recovery reader independent of the main shell are first-class deliverables.

VM storage need not be forced into fine-grained document objects. A large virtual disk can use a block-image backend whose checkpoints are referenced by activity records. The document store and disk backend must agree on checkpoint boundaries without pretending that either understands the guest's internal application transactions.

6.3 Retention, encryption, and deletion

Removing a collection reference does not immediately erase content. Garbage collection must account for snapshots, open capabilities, backups, and explicit retention pins. Storage quotas and a clear retention policy are required; version history is not free storage.

The first design should deduplicate only within an appropriate trust/key domain. Encryption and content addressing must be composed deliberately so that hashes do not become an avoidable cross-domain content oracle. Key separation, authenticated encryption, and metadata exposure belong in the storage threat model, not in a later cosmetic layer.

Versioning reduces the consequences of mistakes and some destructive edits, but an application may still misuse everything it has been authorised to modify. It must not be able to remove protected historical roots merely because it can write the current document.

Local rollback is not a backup. JANUS needs independently retained, encrypted backup copies and tested restore procedures. Deletion policy must explain what remains on replicas and removable media; cryptographic erasure depends on removing all relevant key copies and does not erase plaintext already exported elsewhere.

7. Continuity without magical snapshots

7.1 Three persistence contracts

A world declares one of three initial contracts. **Reconstructible** worlds restart from durable application state. **Checkpointable** worlds can capture execution state under a documented device and software configuration. **Ephemeral** worlds deliberately discard runtime state and may retain only explicitly committed outputs.

An activity record stores the selected worlds, document revisions, presentation layout, and reconstruction instructions. It must distinguish “return to this activity” from “resume the exact CPU instruction”. The former is the preferred baseline because it can survive more software and hardware changes.

QEMU’s migration framework requires device-state support and compatibility rules. Its VFIO migration documentation describes additional device and driver requirements for assigned hardware. These are reasons to qualify checkpoint support, not to promise transparent suspension of every world. [12, 13]

7.2 External effects do not rewind

A restored process may refer to a dead network connection, a removed USB device, a completed print job, or an expired credential. A snapshot cannot undo an email that has already been delivered. Exactly-once effects require cooperation from the external service; the local OS cannot manufacture that guarantee.

Native services should expose reconnect, rebind, cancel, and reconcile operations. Durable operation identifiers can support idempotent retries where the peer protocol permits them. After an uncertain interruption, the system should report “outcome unknown” rather than automatically repeat an irreversible action.

Distributed activity checkpoints require a coordinated consistency protocol; they are not the sum of unrelated RAM dumps. The first release should reconstruct such relationships from durable records rather than claim transparent whole-machine checkpointing.

7.3 Current authority outranks old state

Restoration must consult the current authority record before rebuilding live capabilities. Old memory may contain stale handle values, but those values cannot resurrect revoked grants. Ordinary world checkpoints must not roll the authority record backwards.

Keeping that record in a different file is not sufficient protection against a rollback of the entire physical disk. Strong anti-rollback requires an appropriate external or hardware-backed trust anchor, or an explicit recovery procedure that treats authority as uncertain and requires fresh approval. This remains a platform decision.

A new world incarnation receives fresh session identifiers and relevant randomness. QEMU’s VM Generation ID mechanism provides a precedent for notifying participating guests about snapshot or template-related changes; legacy guests cannot be assumed to implement it. [14]

Secrets in captured RAM need protection, and some key-handling worlds should not be checkpointed at all. Persistent state must remain useful without becoming a permanent archive of every transient secret the owner ever used.

8. Hardware authority, leases, and handover

8.1 Three access modes

Mediated service access exposes a function—audio playback, a camera stream, or block I/O—without granting the hardware registers. **Selected-device assignment** exposes a particular peripheral through a controlled backend. **Controller or PCI assignment** gives a world direct control of the assigned hardware within the platform’s enforceable limits.

These modes are alternatives with different costs and consequences, not a universal ranking. A historical guest may work better with an emulated adapter than a new physical device for which it has no driver. A physical NIC may improve direct control while bypassing a network broker’s application-level policy.

CPU virtualisation support does not establish safe device assignment. The platform must also provide and correctly configure the necessary DMA and interrupt isolation. Linux VFIO documents IOMMU groups as the relevant isolation granularity, including cases where devices cannot be separated because of their topology. [10]

8.2 A lease is a state machine

A device lease records the device or inseparable group, current owner, permitted operation, recovery path, and release requirements. The intended lifecycle is:

```
AVAILABLE -> RESERVED -> QUIESCING -> ASSIGNED  
ASSIGNED -> REVOKING -> RESETING -> AVAILABLE  
Any uncertain recovery state -> QUARANTINED
```

Before assignment, the broker verifies eligibility, detaches the previous driver as appropriate, establishes the intended DMA mappings, and records ownership. Before reuse, it stops new work, drains or aborts outstanding operations, revokes mappings in a safe device-specific order, resets the device, and checks the result. The sequence must come from a reviewed device/platform contract, not a universal guessed recipe.

Libvirt’s managed host-device assignment offers a useful prototype mechanism for detaching and reattaching supported host devices. It is not evidence that every device can be reset safely or that every guest can tolerate removal. [11]

8.3 Failure is an explicit state

A device that cannot be reset must not be silently returned to a different world. Quarantine may require a bus reset, a machine reboot, or physical intervention; collateral effects on other devices must be reported. Checkpointability and safe hot transfer are separate properties.

An IOMMU constrains relevant DMA access; it does not establish that device firmware is benign, prevent every shared-resource attack, or validate the platform’s implementation. Supported configurations must record firmware, topology, and reset evidence.

The initial JANUS appliance should preserve dedicated host-owned input and a recoverable display path. Single-GPU reassignment and arbitrary hot-swapping can be later experiments, not prerequisites for a dependable owner experience.

9. Presentation and the native-like experience

9.1 Immersion is an operating-system contract

JANUS should support two equally legitimate experiences: several cooperating surfaces in an activity, and an immersive world that occupies the selected display and input seat. A foreground lease identifies the surfaces, input routes, audio policy, and optional device assignments involved. It does not automatically enlarge the world's access to documents or networking.

The early Linux prototype can use a minimal Wayland session and `virt-viewer --kiosk`. The viewer's own manual explicitly distinguishes kiosk presentation from a complete secure kiosk configuration. Its disconnect behaviour is also a viewer event, not proof that the guest has shut down. [15]

Consequently, the lifecycle service must distinguish guest shutdown, display disconnect, guest reboot, viewer failure, and explicit owner withdrawal. Losing the console should offer reconnection or recovery; it must not destroy the guest or silently release still-active hardware.

9.2 A trusted route back

An owner recovery gesture must be handled outside the foreground world. If a USB controller and its keyboard are assigned directly to a guest, a host-level keyboard shortcut on that keyboard cannot be assumed available. JANUS must retain a separate host-owned input device, a trusted physical control, or another documented recovery channel.

Similarly, a GPU assigned to a guest cannot simultaneously be assumed to display trustworthy host overlays. A second display path or a tested reset-and-reclaim procedure is necessary. A complete lock operation must also account for a monitor that a guest drives directly; drawing a lock screen on some other output is insufficient.

The recovery interface should identify the active world through trusted presentation and let the owner withdraw foreground access, revoke a device lease, reconnect a console, or terminate the affected world. Restoring that control must not depend on cooperation from the failed or hostile guest.

9.3 Performance and integration

Mediated display should expose efficient shared buffers where supported, with explicit fences, formats, bounds, and ownership transitions. GPU acceleration is a substantial driver and command-validation problem, not an automatic consequence of a small kernel. A shared GPU service may remain in the relevant confidentiality and availability TCB.

Input latency, frame pacing, scaling, monitor changes, cursor ownership, audio clocks, and device hotplug all contribute to the feeling of a real computer. Preservation profiles additionally need selectable integer scaling, aspect ratio, keyboard mapping, and appropriate display modes rather than compulsory stretching to the panel's resolution.

Clipboard, drag-and-drop, screen capture, accessibility, and automation are cross-world data channels. Each needs scoped authority. Accessibility services may legitimately need broad observation, but that authority should be named and reviewable rather than treated as an invisible exception to the model.

10. Networking, compatibility, and preservation

10.1 Network authority is explicit

Native worlds receive network capabilities rather than an automatic machine-wide network presence. A capability may permit a connection to a named service, an approved protocol gateway, an isolated LAN, or a particular overlay. Name resolution and credentials must follow the same scope. An application should not discover private services merely because it knows a conventional localhost port.

Policy must state where it is enforced. A transport broker can restrict endpoints without understanding an encrypted application conversation. A claim such as “this world may access only one web service” needs a defined identity and enforcement mechanism, not merely an IP allow-list labelled HTTPS. Content-aware mediation introduces additional trusted code and privacy consequences.

Compatibility guests can receive a conventional virtual NIC attached to an isolated network service. Legacy systems should initially have no external network access unless their profile explicitly grants it. A passed-through NIC can bypass host network mediation, so that mode must disclose the lost controls and rely on suitable external network isolation where needed.

10.2 Compatibility is a contained personality

Linux, BSD, Windows, and other operating systems can inhabit worlds without defining the native JANUS interface. Their internal users, filesystems, services, and privilege mechanisms remain inside that world. A guest administrator is not automatically a JANUS administrator.

Host/guest integration should use narrowly scoped agents and channels. Mounting the owner’s entire data store into a guest for convenience would undo much of the design. Selected import/export objects, a bounded clipboard exchange, and explicit device grants are preferable starting points.

10.3 Historical and foreign architectures

Hardware acceleration executes suitable guest instruction sets on compatible host architectures; it does not turn an x86 host into a hardware-accelerated SPARC or PowerPC machine. QEMU’s software translation and machine models provide a separate emulation path. [8]

JANUS profiles should record architecture, machine model, firmware, CPU contract, device configuration, backing-image hashes, and tested software. Labels should distinguish an exact-machine target from a compatible virtual platform and an intentionally enhanced configuration. A CPU feature profile is not a promise of period-accurate timing.

QEMU’s published security policy warns that TCG emulation is not a guest-isolation security guarantee. JANUS should therefore confine the emulator itself as untrusted code; on the Linux prototype, sensitive use may place it inside a restricted outer hardware VM. Emulation compatibility must not be mistaken for containment. [16]

Firmware and guest-software provenance must be recorded separately from technical compatibility. The profile catalogue should describe required user-supplied media and avoid implying that technical emulation grants redistribution rights.

11. Software composition and developer interfaces

11.1 Immutable code, separately managed state

A JANUS bundle contains immutable executable components, declared interface versions, dependency identities, requested capabilities, and state-schema information. Installing a bundle makes it available for instantiation; it does not entitle installation scripts to modify arbitrary system state.

A publisher signature authenticates a particular publication under a configured key. It is not evidence that the program is harmless. Local development bundles must remain possible through an explicit owner-controlled development path, without an online account or mandatory vendor approval.

Updates create new bundle identities. The owner can inspect additional requested authority before accepting a change. Old code may remain available, but safe rollback requires compatible state or a preserved pre-migration copy. Reverting executable bytes does not undo an irreversible schema migration.

Update metadata must resist stale or inconsistent publications and permit key rotation and recovery. The Update Framework provides a relevant established design for roles, signed metadata, version checks, and expiry; choosing an implementation remains separate work. [17]

11.2 A small native contract

The native interface should expose world lifecycle, capability transfer, bounded message channels, explicitly shared memory, object transactions, and service discovery within granted namespaces. Language runtimes may wrap these mechanisms, but no runtime should silently introduce ambient file, network, or environment access.

IPC interfaces need stable schema identifiers, version negotiation, cancellation, timeouts, bounded message sizes, and defined failure outcomes. A typed interface reduces ambiguity; it does not make a malicious peer trustworthy. Receivers validate sizes, ownership, and state transitions at the actual trust boundary.

The first native application should be intentionally modest: a document editor or object browser that demonstrates selection, delegated access, durable edits, suspension, and recovery. Reimplementing a contemporary browser before validating those contracts would obscure the architectural experiment.

11.3 An inspectable personal computer

JANUS should expose the world graph, live grants, resource accounts, service health, and recovery history in both graphical and textual forms. A capability-aware command environment can compose tools through object and channel handles. Conventional shells remain available in compatibility worlds.

Diagnostic authority must be scoped. Logs, crash dumps, memory inspection, tracing, and screenshots may contain private material. They should have retention controls, redaction boundaries, and explicit export actions rather than becoming a covert global data store.

No implementation language is selected by this paper. The decision should examine memory and concurrency safety, ABI interoperability, existing driver code, debugging, formal-tool support, and reviewer competence. Memory-safe implementation is desirable where practical, but language choice alone does not establish correct authority, device sequencing, or durable transactions.

12. Building with LLMs, not depending on them

12.1 The engineering hypothesis

JANUS treats LLM assistance as a proposed way to expand implementation and investigation capacity. Candidate tasks include driver adaptation, parser development, interface bindings, documentation, test scaffolding, fault injection, and exploration of existing code. The project must measure whether assistance reduces total engineering effort once review, debugging, and maintenance are counted.

There is no requirement for an inference engine, assistant process, model download, or remote AI service in a released JANUS system. The runtime behaviour should be determined by inspectable code, configured policy, and user actions. Building with LLM assistance does not make JANUS an “AI operating system”.

12.2 Authority and evidence in the development process

A development agent should operate in a disposable, resource-limited environment containing only the repository, toolchain, fixtures, and credentials necessary for its assigned task. Access to signing keys, owner data, raw disks, and production infrastructure must be separately controlled. Building a driver must not implicitly authorise loading it on a valuable physical machine.

Each proposed change should arrive with a bounded diff, a statement of intended behaviour, applicable invariants, build and test commands, observed results, and unresolved limitations. Generated output must not substitute invented evidence for execution logs. A second model’s agreement is not independent verification of correctness.

Tests should include independent specifications, differential comparison where appropriate, malformed inputs, concurrency stress, and fault injection. When the same agent writes implementation and tests, review must actively look for a shared mistaken assumption. Passing a self-authored test is useful evidence, not a complete oracle.

12.3 Concentrate assurance on boundaries

The highest-assurance work belongs at capability transfer, memory/DMA isolation, boot authority, object commit, grant recovery, and trusted presentation. State machines for revocation and device handover are suitable early candidates for model checking. Small parsers and wire protocols are suitable for systematic fuzzing.

An existing verified kernel may reduce one part of the argument. seL4 documents assumptions involving hardware, boot, low-level mechanisms, and information channels; its proofs cannot simply be relabelled as verification of JANUS drivers, policy, storage, or desktop services. [2]

Releases should retain source identities, compiler and dependency versions, build recipes, test artefacts, and provenance of imported or generated code. Reproducible builds are a target to demonstrate by comparison, not a label earned by writing a build script.

The intended leverage is disciplined: generate more candidate implementations while keeping acceptance evidence explicit and the amount of broadly trusted code controlled. Isolation limits consequences; it does not excuse shipping a defective component that can still destroy the data it legitimately owns.

13. Threat model and operational resilience

13.1 Assets and adversaries

The initial threat model includes hostile applications, compromised compatibility guests, malformed documents and disk images, malicious network peers, faulty or hostile drivers, and peripheral-triggered failures. Protected assets include the owner's data and keys, cross-world confidentiality and integrity, accurate grant decisions, and the ability to regain control of the machine.

The model also includes power loss, exhausted storage, failed updates, and ordinary software crashes. Recovery is part of the security argument: a system whose only repair path grants everything to an untrusted component has not preserved its boundary.

It does not claim complete resistance to compromised platform firmware, invasive physical attacks, every microarchitectural side channel, or arbitrary defective hardware. Shared caches, memory buses, GPU resources, and storage contention complicate strong non-interference. High-sensitivity configurations may need separate cores, devices, or machines; their guarantees require separate analysis.

13.2 Trust follows the protected property

Property	Components that may remain trusted
World memory isolation	Supervisor, platform configuration, relevant firmware and hardware assumptions
Document confidentiality	Key service, object service or encryption endpoint, authorised readers, relevant display path
Correct grant decisions	Owner authentication, trusted grant interface, authority service, IPC enforcement
Continued owner control	Scheduler, reserved recovery services, trusted input path, recoverable display
Update integrity	Signing policy, metadata verification, installer/boot path, recovery mechanism

This table is a starting point for per-configuration analysis, not a complete proof. A compromised storage driver can still deny service; a compromised authorised editor can leak a document through any channel it legitimately possesses. An isolated component is not necessarily an unimportant component.

13.3 Failure and recovery policy

Driver restart should be a local operation where hardware and protocol state permit it. Shared services need bounded restart loops, health checks, and escalation to a safe degraded state. If a GPU service fails, applications may continue computing while presentation requires recovery; claiming that the “desktop continues unaffected” would conceal the dependency.

The system image should support a known-good recovery environment independent of the current personal graph. The authority/key bootstrap, minimum storage driver, and recovery reader must be obtainable without first starting every persistent service, avoiding a circular boot dependency.

Updates should preserve a recoverable system image and the required data-schema boundary. Power-failure tests must interrupt commits and updates at multiple points. Logs should record what was committed, what was abandoned, and what remains uncertain without exposing secrets unnecessarily.

An owner-requested full administrative or debugging mode may intentionally relax isolation. That transition should be authenticated, visible, and recorded, with its consequences stated plainly rather than obscured behind a claim that JANUS has no privileged state.

14. Implementation strategy and milestones

14.1 Prototype the contracts before the kernel

The first implementation should validate the personal-computing model on an existing host. A practical candidate is Linux, KVM/QEMU, libvirt, a minimal Wayland session, and a JANUS shell plus bounded management service. QEMU and libvirt provide existing machine/device configuration mechanisms, and virt-viewer provides a display frontend. [8, 11, 15]

This hosted system remains dependent on Linux and its privileged services. It is a behavioural prototype, not proof of the final supervisor's isolation model. Its purpose is to stabilise manifests, activity records, grant UX, recovery semantics, and test fixtures before replacing the protection substrate.

The user-facing shell must not accept arbitrary command strings and forward them to privileged QEMU or libvirt interfaces. The management service should accept bounded operations against validated profiles and already authorised resources. QEMU's monitor is a privileged interface, and its security guidance stresses least-privilege deployment. [16]

14.2 Evidence-gated development

Stage	Deliverable	Exit evidence
M0 — Contracts	World/object/authority schemas and threat model	Reviewed invariants, executable state-machine tests, versioned decisions
M1 — Hosted appliance	Select, run, recover, and leave a guest activity	Console loss differs from shutdown; owner recovery remains available
M2 — Native experience	Small native tool, object grants, activity reconstruction	No ungranted object access; interrupted commits and stale grants tested
M3 — Substrate experiment	Same narrow contracts on a capability substrate	Two isolated worlds, service IPC, resource accounting, documented TCB
M4 — Hardware ownership	One supported assignable device class	Repeated assignment/revocation/reset tests; quarantine on uncertainty
M5 — Personal alpha	Daily-use activity with updates and backup	Restore rehearsal, regression suite, measured responsiveness and power

Milestones are defined by evidence, not dates. M3 should compare reuse of Genode/Sculpt components, a seL4-based composition, and any justified custom substrate work. The choice must account for drivers, development tools, virtualisation support, licence obligations, and assurance—not kernel line count alone. [1, 3, 4]

14.3 Reference hardware

Select one physical reference machine after inventorying its CPU features, firmware settings, IOMMU topology, storage, display, and input devices. A known-compatible display path and separable experimental peripheral are more valuable initially than a broad unsupported hardware list.

Disposable VM tests precede physical tests, but successful emulated boot does not establish suspend, power management, DMA isolation, or device reset on the real machine. Preserve the existing host installation and use explicit recovery media or a separate test disk. No machine should become a destructive test target merely because it appears in a hardware inventory.

15. Evaluation and falsifiable claims

JANUS needs an evaluation plan that can reject attractive ideas. The baseline should include a conventional desktop and the hosted JANUS prototype on the same hardware where meaningful. Later native implementations must disclose different drivers, services, compiler settings, and power policy rather than attribute every difference to the kernel model.

15.1 Functional and isolation tests

The central authority test is simple: a world given one object cannot open another by discovering its identifier, guessing a path, replaying a handle, or invoking a confused deputy. Delegation and revocation must be tested during concurrent operations, service restart, checkpoint restoration, and software update.

A device test should attempt out-of-contract access in an authorised laboratory configuration and verify the intended confinement. Reset failures must leave the device quarantined. A foreground test should make the active world unresponsive and confirm that the owner can still reach trusted controls without that world's cooperation.

15.2 Measurement plan

Hypothesis	Measurement	Evidence that would challenge it
Worlds feel immediate	Input-to-display latency; frame-time tails; audio underruns under background load	Frequent foreground stalls or disrupted audio
Isolation is affordable	Total memory; service CPU; IPC cost; idle and active power	Per-world costs make ordinary activities impractical
Activities are recoverable	Power-loss commit tests; reconstruction success; time to usable activity	Lost committed work, stale authority, or repeated external effects
Device leases are dependable	Transfer/reset outcomes across supported device/firmware combinations	Unsafe reuse or recovery requiring undocumented intervention
Grants are understandable	Task-based owner studies; incorrect grants; ability to identify active authority	Users cannot predict which data or devices an app may access
LLM assistance helps engineering	Reviewed change throughput; escaped defects; review and rework effort	Code volume grows while maintenance and defect costs worsen

Measurements should report workload definitions, repeated-run distributions, percentile latency, and failures. Averages alone hide the pauses that make a home computer feel unreliable. Targets must be set before acceptance testing and tied to a named reference configuration; this paper supplies no invented boot-time, memory, or throughput result.

15.3 Adversarial recovery exercises

Required exercises include storage-full during commit, power loss during update, a corrupted activity root, an unavailable backup, a revoked capability inside a restored checkpoint, a missing USB device, a failed display frontend, and a guest retaining hardware after its console disappears.

Performance optimisations that weaken isolation must be evaluated as explicit configuration changes. Shared caches, shared GPU state, deduplication domains, and unbounded common services cannot be treated as free improvements.

The minimum success criterion is a useful complete activity with demonstrable boundaries and a dependable recovery path. A fast boot animation or a guest displayed fullscreen is not sufficient evidence that JANUS's architectural claims hold.

16. Decisions required and conclusion

16.1 Open engineering decisions

The following choices require an explicit architecture decision record before implementation depends on them. The provisional positions are suggestions for experiments, not decisions attributed to the author.

Decision	Provisional direction	Evidence or approval needed
Native substrate	Evaluate an existing capability framework/kernel before starting a new one	Driver/VM prototype, assurance boundaries, maintenance assessment
First physical target	One documented x86-64 configuration, subject to inventory	Actual isolation topology, recovery display/input, reliable storage
Implementation languages	Keep the choice open; favour reviewable boundaries and safe abstractions	ABI/toolchain trials, existing code requirements, reviewer capability
Persistence baseline	Durable objects plus activity reconstruction first	Crash consistency, migration and export tests
Graphics and device policy	Mediated display by default; opt-in assignment of supported devices	Latency measurements, reset and trusted-recovery evidence
Ownership and recovery	Explicit local owner authority, protected key service, offline recovery	Authentication design, rollback threat model, recovery-key policy
Distribution and licensing	No licence or redistribution policy selected here	Author decision and review of all incorporated dependencies

A future expansion to additional host architectures, multiple interactive seats, or remote world execution should preserve these contracts where possible. It should not be allowed to delay the first coherent local system or silently weaken the initial single-owner assumptions.

16.2 Conclusion

JANUS is a proposal to make the personally owned computer a composition of explicitly bounded worlds. Its organising ideas are not the removal of every familiar interface or the elevation of VMs into a universal implementation rule. They are visible authority, persistent activities, accountable resources, and a controlled relationship between software and physical hardware.

The strongest experience would let an owner move naturally between a native document tool, a development environment, and an immersive historical or contemporary operating system. Each would receive only the data, services, and devices deliberately granted to it. A failure would have a defined scope, and recovery would preserve both the owner's work and the current authority boundary.

The proposed development process uses LLMs to assist construction while refusing to equate generated code with validated code. Its leverage depends on better interfaces, smaller reviewable changes, independent evidence, and isolation that contains mistakes—not on an AI making operating-system decisions at runtime.

JANUS should make the computer feel wholly personal without making every program wholly trusted. The next architectural advance is a demonstrable implementation of that contract, beginning with a small useful activity and an owner who can always regain control.

Appendix A. Illustrative world contract

The following is design notation, not a supported configuration format or executable JANUS API. Symbolic object names are resolved through authorised brokers; none of the strings is itself a bearer capability. Requested authority is distinct from granted authority.

```

schema: janus.world/v0-draft
identity: example.editor
execution:
  class: native-domain
  component: bundle:example-editor/pinned-revision
resources:
  memory_max: 256 MiB
  cpu_class: interactive
  queue_limit: 128
requests:
  - interface: display.surface
    scope: current-activity
  - interface: input.focused
    scope: own-surface
  - interface: objects.pick
    rights: [read, create-revision]
  - interface: objects.private-state
    scope: own-instance
network: none
persistence:
  contract: reconstructible
  state_schema: example.editor-state/v1
  restore_authority: revalidate-current-grants

```

A compatibility profile additionally identifies its machine and device model, emulator/VMM build, firmware identity, disk revision, and supported persistence mode. Device passthrough is a separately approved lease, not an automatic consequence of selecting the profile.

Lifecycle invariants

```

DECLARED -> INSTANTIATING -> RUNNING -> QUIESCING
QUIESCING -> SUSPENDED or STOPPED
SUSPENDED -> RESTORING -> RUNNING
Any execution state -> FAULTED -> RECOVERY or STOPPED

```

The manager must distinguish suspension from durable checkpoint completion. A restore creates a new incarnation and revalidates authority before external I/O. Failed restoration leaves the committed source state available. Neither a display disconnect nor an expired foreground lease, by itself, proves that execution has stopped.

Proposed invariant: effective authority is bounded by the current valid grants, not by the privileges recorded in an old manifest or memory image. Once revocation completes, replayed references cannot authorise new operations. Effects already committed outside that boundary require their own reconciliation rules.

Appendix B. Terminology and claim boundaries

Term	Meaning in this paper
Activity	An owner-facing grouping of objects, worlds, and presentation state; not automatically one security domain.
World	A bounded execution and authority domain with an identity, resources, lifecycle, and persistence contract.
Native domain	A world using the native JANUS service contract and ordinary hardware memory protection, without a required guest OS.
Compatibility world	A world containing an existing operating system and its internal conventions.
Capability	An enforced, unforgeable reference conveying specific authority; not merely an object name or hash.
Grant	A recorded authorisation from which live capabilities or service access may be established.
Lease	Time- or lifecycle-bounded ownership/access governed by an explicit withdrawal and recovery protocol.
Object identity	A stable reference to an entity across revisions, distinct from content identity and authority.
Content identity	An identifier for immutable content, commonly hash-derived; it does not authorise access.
Reconstruction	Restarting a world from durable application state and rebuilding valid service relationships.
Checkpoint	Captured execution/device state whose restoration has specified compatibility and consistency requirements.
Incarnation	One execution lifetime of a persistent world identity; renewed on relevant restart/restore operations.
TCB	Trusted computing base: the components whose behaviour must be correct for a particular protected property.
Trusted path	An input/output and authentication route the affected untrusted world cannot impersonate or disable within the stated threat model.
Quarantine	A state in which a device or component is withheld from reuse because safe recovery is not established.

What this draft does not assert

It does not assert that all worlds require hardware virtualisation, that there is no privileged code, that persistence is free, that an IOMMU makes every device safe, or that all devices can be hot-transferred. It does not claim that a guest's historical CPU label implies cycle accuracy, that a verified kernel verifies the whole desktop, or that LLM assistance guarantees faster or better engineering.

Version 0.1 is an architecture discussion draft. Statements about JANUS describe intended behaviour and research work unless explicitly identified as existing external mechanisms. Changes to the proposed contracts should be recorded together with rationale, alternatives, and the evidence that would justify revisiting them.

References

Primary project documentation consulted on 20 September 2026. Online documents, especially development-branch manuals, can change; implementation work must pin exact upstream revisions. References establish precedents and documented mechanisms, not measured JANUS results.

[1] seL4 Project. **Fact Sheet**. Capability-based kernel features and hypervisor role.

<https://sel4.systems/About/fact-sheet.html>

[2] seL4 Project. **What the Proofs Assume**. Scope, hardware assumptions, boot and information-channel qualifications.

<https://sel4.systems/Verification/assumptions.html>

[3] Genode Labs. **General overview**. Component organisation, resource delegation, capabilities, and application-specific trusted computing bases.

<https://genode.org/documentation/general-overview/>

[4] Genode Labs. **Sculpt OS**. Published system overview and documentation entry point.

<https://genode.org/download/sculpt>

[5] Qubes OS Project. **Architecture**. Virtual-machine compartmentalisation and desktop integration.

<https://doc.qubes-os.org/en/latest/developer/system/architecture.html>

[6] CapROS Project. **The Capability-based Reliable Operating System**. Capability architecture, orthogonal persistence, and relationship to EROS.

<https://www.caproos.org/>

[7] University of Cambridge, Systems Research Group. **Nemesis**. Resource accounting and support for time-sensitive applications. Historical project overview.

<https://www.cl.cam.ac.uk/research/srg/netos/projects/archive/nemesis/>

[8] QEMU Project. **System Emulation: Introduction**. Accelerators, software CPU emulation, machine models, devices, and management interfaces.

<https://www.qemu.org/docs/master/system/introduction.html>

[9] Linux Kernel Project. **The Definitive KVM API Documentation**. VM and vCPU creation, execution, and feature discovery.

<https://docs.kernel.org/virt/kvm/api.html>

References (continued)

[10] Linux Kernel Project. **VFIO — Virtual Function I/O**. DMA isolation, IOMMU groups, and device assignment boundaries.

<https://docs.kernel.org/driver-api/vfio.html>

[11] libvirt Project. **Domain XML format**. Domain configuration, host devices, graphics, storage, and network interfaces.

<https://libvirt.org/formatdomain.html>

[12] QEMU Project. **Migration framework**. Device-state and compatibility requirements for migration/save-state mechanisms.

<https://www.qemu.org/docs/master/devel/migration/main.html>

[13] QEMU Project. **VFIO device migration**. Requirements and limitations for migration involving assigned devices.

<https://www.qemu.org/docs/master/devel/migration/vfio.html>

[14] QEMU Project. **Virtual Machine Generation ID Device**. Notification of snapshot/template-related changes to participating guests.

<https://www.qemu.org/docs/master/specs/vmgenid.html>

[15] virt-viewer Project. **virt-viewer manual, upstream source**. Kiosk mode, disconnect behaviour, and the limits of viewer-only lockdown.

<https://gitlab.com/virt-viewer/virt-viewer/-/raw/master/man/virt-viewer.pod>

[16] QEMU Project. **Security**. Supported isolation use cases, the TCG qualification, least-privilege deployment, and monitor authority.

<https://www.qemu.org/docs/master/system/security.html>

[17] The Update Framework. **Specification**. Signed metadata roles, version and expiration checks, and update trust management.

<https://theupdateframework.github.io/specification/latest/>

JANUS — Architecture Whitepaper, v0.1

Danyal A. Samak · dabsamak@tuta.com · <https://www.cryogenix.org>

20 September 2026

Editorial disclosure: prepared with LLM-assisted drafting and technical-source checking for author review. Proposed engineering choices and all JANUS-specific claims remain subject to validation. This draft does not apply a software or document redistribution licence.